

Assignment 2 Report – Perceptron

Tavonga Tawonezvi

1. Perceptron Implementation and Data Processing

BinaryPerceptron: handles binary classification, perceptron learning rule, `predict()` returns 0/1.

MultiClassPerceptron: wraps 10 BinaryPerceptron instances (one-vs-rest). `predict()` returns label in `{0, ..., 9}` corresponding to the max score.

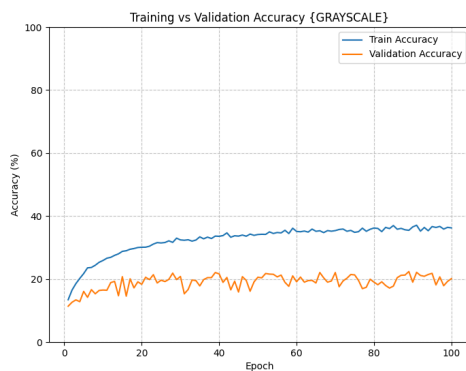
Some key design choices:

1. Modular design: BinaryPerceptron is reusable.
2. Training loop applies perceptron update rule per sample.
3. Used an 80/20 split for the held-out validation set

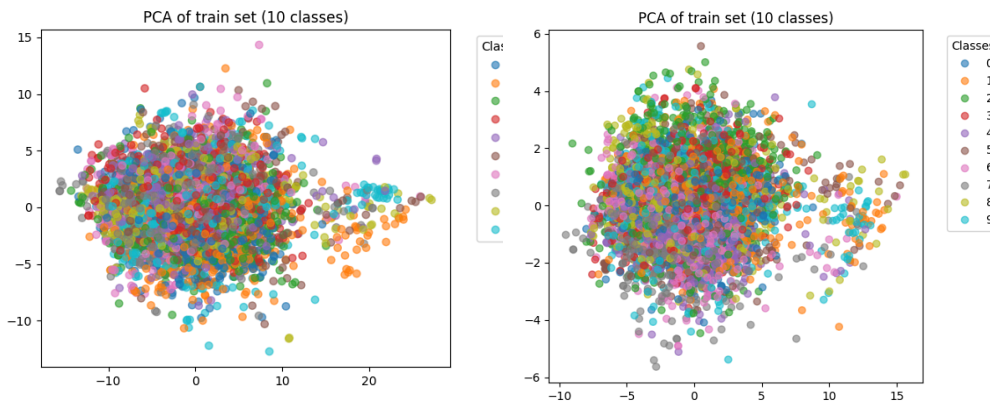
2. Training

The MultiClassPerceptron object can be created with the ability to choose to train it on the rgb or grayscale images as a parameter. The training data was also shuffled between epochs to prevent the perceptron from learning patterns in the order of samples and to improve generalization.

The perceptron did not seem to converge when training. While training accuracy did improve after each epoch, validation accuracy oscillated at about 20% during training(see *fig1 below*).



This seemed to suggest overfitting (model learning training data too well), however after plotting the PCA of the 10 characters in the training set(gray scale and rgb), it showed that it was because the data was likely not linearly separable for both rgb and grayscale images(see *fig2 and fig2.1 below*). As shown in the plots below, there are no clear, separate clusters that form for each class in both rgb and grayscale formats.



3. Comparing early stop criteria

1. Max number of epochs - I tested the model with a range of max_epochs, keeping the learning rate constant. I decided to keep max_epochs at 100 iterations, since training accuracy began to plateau here for all runs for both grayscale and rgb images(see fig 3).
2. Early stopping on validation accuracy - Validation accuracy never seemed to improve to get close to training accuracy, even after tuning hyperparams(reasons for this are given further down). I chose this early stopping method to get the best possible model. Training would stop after validation accuracy stopped improving for 30 or more epochs.

4. Hyperparameter Tuning

The grid search below was used to test different combinations of hyperparameters, weight inits, as well as max_epochs and a suitable early stopping count:

```
learning_rates = [0.01, 0.02, 0.05, 0.08, 0.1, 0.4]
max_epochs_list = [50, 80, 100, 120, 150]
val_error_threshold = [10, 20, 35, 45]
weight_inits = ["zeros", "constant", "gaussian", "uniform"]
```

Findings were as follows:

1. **GRAYSCALE:**

Weight init: zeros: 0.1825, uniform: 0.1925, constant<0.5>: 0.1981

Different weight init methods did not improve convergence significantly. I chose the constant init since it had the best accuracy score keeping a constant hyperparam.

Input normalization: scaled to [0,1] for training stability since image pixels are [0,255]; without scaling, updates would be too large. This also ensures weights can be interpreted as importance per pixel intensity during training.

Z-score normalization assumes the data follows a bell-shaped distribution. Image pixels are bounded [0,255] and are often highly skewed because of backgrounds. Therefore, min-max scaling was more appropriate for this data.

Learning rate: Learning rates that were too low(0.001) stopped training early since validation accuracy was not improving. Ones that were too high(0.05) trained the model for longer without actually making any progression in validation accuracy. 0.02 showed the most stability.

2. RGB:

Weight init: Because of the extra dimension(colour channel), I used random init to prevent bias towards a single color channel at start

Input normalization: Scaled to [0,1] - crucial since RGB ranges up to 255 and without scaling updates would be too large. Z-score normalization: same argument as in grayscale images normalization

Learning rate: Slightly higher than for grayscale(0.06) - this worked best the RGB images likely because it helps compensate for higher dimensionality and slower convergence.

5. Evaluation and insights

Class Key: *bart_simpson*: 0 *lisa_simpson*: 1 *homer_simpson*: 2 *marge_simpson*: 3 *ned_flanders*: 4

milhouse_van_houten: 5 *principal_skinner*: 6 *charles_montgomery_burns*: 7
moe_szyslak: 8 *krusty_the_clown*: 9

1. Final model metrics { held-out validation set }:

Confusion Matrix: {GRAYSCALE}

```
[[32  5 40  3 16  3 25  2  3 31]
 [27 13 34  4 19  8 19  5  1 30]
 [16  5 53  1 17  1 17  7  4 39]
 [18  5 50  4 16  8 19  4  1 35]
 [14  3 22  4 38  8 30  6  6 29]
 [26  6 34  5 16 22 15  2  3 31]
 [15  5 25  3 25  7 51  6  1 22]
 [20  3 24  0 26  5 23 29  3 27]
 [21  4 21  5 15  9 21 10 22 32]
 [11  7 32  4 12  8 21  7  5 53]]
```

Classification Report: {GRAYSCALE}

	precision	recall	f1-score	support
0	0.16	0.20	0.18	160
1	0.23	0.08	0.12	160
2	0.16	0.33	0.21	160
3	0.12	0.03	0.04	160
4	0.19	0.24	0.21	160
5	0.28	0.14	0.18	160
6	0.21	0.32	0.25	160
7	0.37	0.18	0.24	160
8	0.45	0.14	0.21	160
9	0.16	0.33	0.22	160

accuracy			0.20	1600
macro avg	0.23	0.20	0.19	1600
weighted avg	0.23	0.20	0.19	1600

Accuracy: 0.1981
Precision: 0.2334
Recall: 0.1981
F1 Score: 0.1874

Confusion Matrix: {RGB}

```
[[ 69 52  8  3 11  8  1  3  0  5]
 [ 27 101  9  3 10  5  0  2  0  3]
 [ 31 49 45  1 17  6  0  1  1  9]
 [ 10 22 11 82 14 13  2  0  0  6]
 [ 37 37  6  2 61  4  2  5  2  4]
 [ 22 51  6  2 10 55  0  0  0 14]
 [ 32 35 12  5 30 10 18  8  1  9]
 [ 34 35 13  2 33 12  6 19  1  5]
 [ 32 44 22  0 18  9  0  4 14 17]
 [ 19 22 11  6 10  7  0  0  1 84]]
```

Classification Report: {RGB}

	precision	recall	f1-score	support
0	0.22	0.43	0.29	160
1	0.23	0.63	0.33	160
2	0.31	0.28	0.30	160
3	0.77	0.51	0.62	160
4	0.29	0.38	0.33	160
5	0.43	0.34	0.38	160
6	0.62	0.11	0.19	160
7	0.45	0.12	0.19	160
8	0.70	0.09	0.16	160
9	0.54	0.53	0.53	160

accuracy			0.34	1600
macro avg	0.46	0.34	0.33	1600
weighted avg	0.46	0.34	0.33	1600

Accuracy: 0.3425
Precision: 0.4557
Recall: 0.3425
F1 Score: 0.3310

2. Final model metrics { test set }:

Making predictions on test set...

Confusion Matrix: {GRAYSCALE}

```
[[165  5  0  0  6  2  1 15  1  5]
 [109  4  1  5  8  3  3 52  1 14]
 [156  8  3  1  7  1  2  7  0 15]
 [113  3  3  1  5  1  2 30  7 35]
 [159 12  1  2  4  1  1 16  0  4]
 [152  4  0  7  4  2  1 15  3 12]
 [138 11  0  4  2 13  1 17  1 13]
 [112 10  1  4  8  4  4 29 14 14]
 [128  4  0  0 16  2  8 21  3 18]
 [136  5  1  1 16  2  9 19  0 11]]
```

Classification Report: {GRAYSCALE}

	precision	recall	f1-score	support
0	0.12	0.82	0.21	200
1	0.06	0.02	0.03	200
2	0.30	0.01	0.03	200
3	0.04	0.01	0.01	200
4	0.05	0.02	0.03	200
5	0.06	0.01	0.02	200
6	0.03	0.01	0.01	200
7	0.13	0.14	0.14	200
8	0.10	0.01	0.03	200
9	0.08	0.06	0.06	200
accuracy			0.11	2000
macro avg	0.10	0.11	0.06	2000
weighted avg	0.10	0.11	0.06	2000

Accuracy: 0.1115
Precision: 0.0979
Recall: 0.1115
F1 Score: 0.0561

Confusion Matrix: {RGB}

```
[[ 74  87  10  1 13  6  1  3  0  5]
 [ 38  58  16  3 30 10  5 16  1 23]
 [ 41  54  62  4 18  8  2  1  1  9]
 [ 21  21  7  2 19 15  1  4  1 109]
 [ 31 122  9  5 12  7  0  1  0 13]
 [ 12  33  7 117 12 11  0  1  0  7]
 [ 37  46 13  4  6 73  5  2  1 13]
 [ 41  52 19  0 28 15  3  6 12 24]
 [ 35  44 11  1 89  8  2  1  0  9]
 [ 35  53 15  1 53  6 23  5  1  8]]
```

Classification Report: {RGB}

	precision	recall	f1-score	support
0	0.20	0.37	0.26	200
1	0.10	0.29	0.15	200
2	0.37	0.31	0.34	200
3	0.01	0.01	0.01	200
4	0.04	0.06	0.05	200
5	0.07	0.06	0.06	200
6	0.12	0.03	0.04	200
7	0.15	0.03	0.05	200
8	0.00	0.00	0.00	200
9	0.04	0.04	0.04	200
accuracy			0.12	2000
macro avg	0.11	0.12	0.10	2000
weighted avg	0.11	0.12	0.10	2000

Accuracy: 0.1190
Precision: 0.1103
Recall: 0.1190
F1 Score: 0.1001

RGB vs grayscale analysis

Looking at the validation set:

- Weak in both greyscale and rgb formats: Lisa (1), like Bart(0), has spiky hair; in grayscale, their silhouettes are almost identical, so the perceptron confuses them. Even in RGB, both have yellow skin, making colour less useful as an extra feature.
- Skinner (6) looks visually generic (plain hair, suit); lacks bold colours or distinctive shapes, so his pixels don't strongly separate him from Moe(8) or Burns(7)
- Moe (8) visually overlaps with Skinner(6) and Burns(7), especially in grayscale where clothing/skin/hair pixels are at very similar tones.
Marge (3): In grayscale had very poor recall, but in RGB, her performance improved to ~32% recall, likely because of her bright blue hair, which provides a strong separable colour feature.
- Krusty (9): In RGB, Krusty achieved the best recall (~53%), as his coloured clown costume and green hair make him highly distinctive. In grayscale, these cues collapsed, leaving him harder to separate.

Conclusion:

Both models perform very poorly overall on test data. Accuracy collapsed to ~10% for the 10 classes, which is equivalent to barely random guessing. RGB shows promise for characters with strong color cues (Marge, Krusty, Milhouse), but the perceptron isn't powerful enough to consistently leverage that advantage. This is because the data isn't linearly separable and a more sophisticated model that doesn't rely on data being linearly separable should be used.